



# DOCUMENTO TÉCNICO

CARRERA DE RELEVOS CON ROBOTS EN  
COPPELISAIM USANDO ROS2

CARRERA DE RELEVOS CON ROBOTS EN COPPELIASIM  
USANDO ROS2  
ROBÓTICA MÓVIL

*Ana Blázquez, Lucía Pardo, Mario Sebastián y Carla Soler.*

## ÍNDICE

|                              |   |
|------------------------------|---|
| OBJETIVO DEL PROYECTO .....  | 2 |
| REQUISITOS DEL SISTEMA.....  | 2 |
| INSTALACIÓN .....            | 3 |
| EJECUCIÓN DEL PROYECTO ..... | 4 |
| DESCRIPCIÓN TÉCNICA .....    | 5 |
| VALIDACIÓN Y PRUEBAS .....   | 7 |

## ÍNDICE FIGURAS

|                                                           |   |
|-----------------------------------------------------------|---|
| Figura 1 Consola al ejecutar waypoints_node .....         | 4 |
| Figura 2 Robots clonados.....                             | 5 |
| Figura 3 Robots en la escena .....                        | 6 |
| Figura 4 Rqt_graph con los nodos y topics .....           | 7 |
| Figura 5 Robot1 moviéndose por la ruta.....               | 8 |
| Figura 6 Robot2 comenzando a moverse tras el relevo ..... | 8 |

## OBJETIVO DEL PROYECTO

Este proyecto tiene como finalidad desarrollar y simular una **carrera de relevos entre dos robots móviles** utilizando el modelo **Turtlebot3**. Esta simulación se lleva a cabo en un entorno virtual creado con **CoppeliaSim**, y los robots son controlados mediante el sistema operativo robótico **ROS2 (versión Humble)**.

La dinámica del proyecto consiste en que el **primer robot recorra un trayecto predefinido**, siguiendo una serie de waypoints. Una vez alcanzado el final del recorrido, el robot detiene su movimiento y **activa al segundo robot**, el cual repite el mismo trayecto desde el inicio.

Este ejercicio pone en práctica conceptos fundamentales de la robótica móvil, como la navegación basada en waypoints, el control mediante el algoritmo **Pure Pursuit**, la sincronización entre robots, el uso de transformaciones espaciales con TF2 y la integración de ROS2 con simuladores. Además, permite experimentar con la comunicación entre nodos y la coordinación en tareas colaborativas dentro de un entorno simulado.

Se pretende aplicar técnicas como:

- Control de trayectoria mediante *cmd\_vel* y *odom*
- Sincronización entre robots usando nodos coordinadores
- Comunicación entre nodos con topics ROS2
- Configuración dinámica con parámetros y remapeo
- Integración de entorno de simulación realista

## REQUISITOS DEL SISTEMA

- **Sistema operativo:** Ubuntu 22.04 LTS
- **ROS2 Humble**
- **CoppeliaSim:** Versión 4.6 o superior
- **Python:** 3.10
- **Dependencias ROS2:**  

```
sudo apt install ros-humble-turtlebot3 \
ros-humble-turtlebot3-simulations \
ros-humble-rqt \
ros-humble-rqt-graph \
ros-humble-rviz2
```

# INSTALACIÓN

## 1. Instalación

Sigue estos pasos para configurar tu entorno y compilar el proyecto:

1. Clonar Repositorio: Abre una terminal y clona el repositorio del proyecto.

```
git clone https://github.com/larmesto/RM_prac
```

2. Clonar o descargar el fichero trabajo\_pkg dentro de *RM\_prac/src*

```
cd ~/RM_prac/src/trabajo_pkg # Asegúrate de que el paquete está dentro de src
```

```
cd ~/RM_prac # Vuelve a la raíz del workspace
```

3. Compilar el Proyecto: Desde la raíz de tu workspace (*~/RM\_prac*), compila el paquete *trabajo\_pkg*. Es recomendable realizar una limpieza completa antes de la compilación para evitar problemas de versiones anteriores.

```
rm -rf build install log # Limpia compilaciones anteriores
```

```
colcon build --symlink-install --packages-select trabajo_pkg
```

4. Fuente del Entorno: Añade las siguientes líneas al final de tu archivo *~/.bashrc* para cargar el entorno de ROS2 y tu workspace automáticamente cada vez que abras una nueva terminal. Luego, aplica los cambios.

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

```
echo "source ~/RM_prac/install/setup.bash" >> ~/.bashrc
```

```
echo "export TURTLEBOT3_MODEL=Burger" >> ~/.bashrc # Si usas TurtleBot3
```

```
source ~/.bashrc
```

*Nota: Si ya tienes estas líneas, no es necesario duplicarlas. Asegúrate de que la ruta *~/RM\_prac/install/setup.bash* sea correcta para tu workspace.*

5. Ejecutar CoppeliaSim: Asegúrate de tener CoppeliaSim instalado en tu sistema. La ruta por defecto en el launch file es */home/mario/CoppeliaSim\_Edu\_V4\_9\_0\_rev6\_Ubuntu22\_04/coppeliaSim.sh*. Si tu instalación es diferente, deberás modificar el archivo *relevos\_launch.py* para apuntar a la ruta correcta del ejecutable de CoppeliaSim.

```
# En relevos_launch.py
```

```
coppelasim_executable = LaunchConfiguration('coppelasim_exec', \
default='/home/mario/CoppeliaSim_Edu_V4_9_0_rev6_Ubuntu22_04/coppeliaSim.sh')
```

No es necesario ejecutarlo manualmente en este punto, ya que el launch file lo iniciará.

## EJECUCIÓN DEL PROYECTO

Una vez que la instalación y compilación se han completado, puedes ejecutar el proyecto siguiendo estos pasos:

1. Fuente del Entorno: Abre una nueva terminal y asegúrate de que el entorno de ROS2 y tu workspace están cargados. Esto se hace automáticamente si seguiste el paso 1.4 de la instalación. Si no, ejecútalo manualmente:

```
source ~/RM_prac/install/setup.bash
```

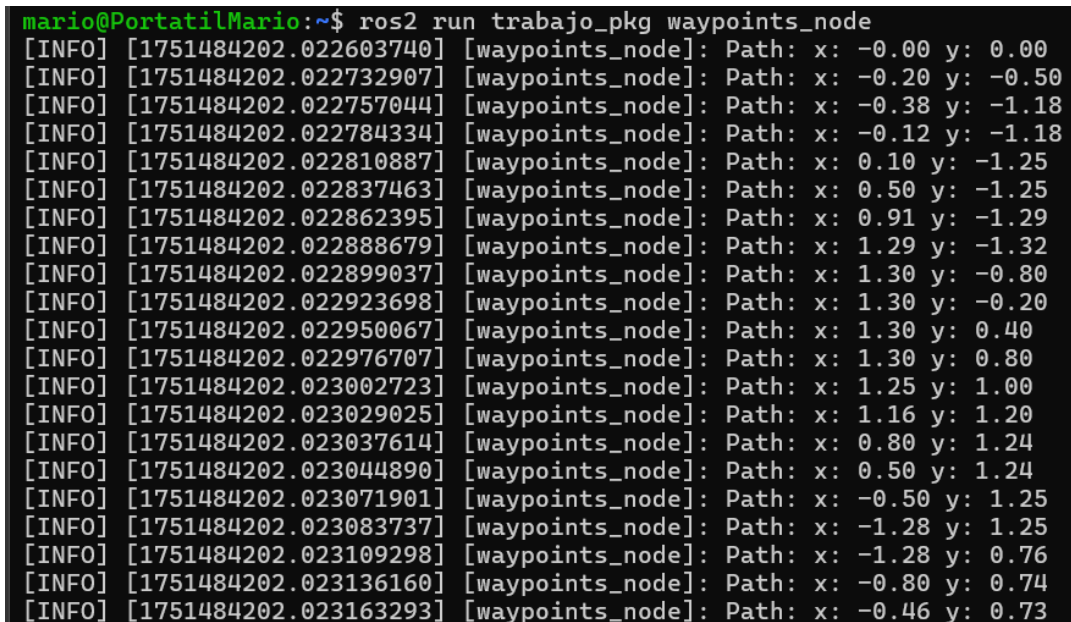
2. Ejecutar los Nodos con Launch: Para iniciar la simulación de Coppeliasim y todos los nodos de control de los robots y el gestor de relevos, utiliza el archivo relevos\_launch.py.

```
ros2 launch trabajo_pkg relevos_launch.py
```

Esto iniciará Coppeliasim con la escena relevos.ttt y los nodos pure\_pursuit\_robot1, pure\_pursuit\_robot2 y relay\_manager.

3. **Publicar Waypoints (en una terminal separada):** El nodo waypoints\_node es responsable de publicar la trayectoria que seguirán los robots. Necesitas ejecutarlo en una terminal separada después de lanzar el sistema principal.

```
ros2 run trabajo_pkg waypoints_node
```



```
mario@PortatilMario:~$ ros2 run trabajo_pkg waypoints_node
[INFO] [1751484202.022603740] [waypoints_node]: Path: x: -0.00 y: 0.00
[INFO] [1751484202.022732907] [waypoints_node]: Path: x: -0.20 y: -0.50
[INFO] [1751484202.022757044] [waypoints_node]: Path: x: -0.38 y: -1.18
[INFO] [1751484202.022784334] [waypoints_node]: Path: x: -0.12 y: -1.18
[INFO] [1751484202.022810887] [waypoints_node]: Path: x: 0.10 y: -1.25
[INFO] [1751484202.022837463] [waypoints_node]: Path: x: 0.50 y: -1.25
[INFO] [1751484202.022862395] [waypoints_node]: Path: x: 0.91 y: -1.29
[INFO] [1751484202.022888679] [waypoints_node]: Path: x: 1.29 y: -1.32
[INFO] [1751484202.022899037] [waypoints_node]: Path: x: 1.30 y: -0.80
[INFO] [1751484202.022923698] [waypoints_node]: Path: x: 1.30 y: -0.20
[INFO] [1751484202.022950067] [waypoints_node]: Path: x: 1.30 y: 0.40
[INFO] [1751484202.022976707] [waypoints_node]: Path: x: 1.30 y: 0.80
[INFO] [1751484202.023002723] [waypoints_node]: Path: x: 1.25 y: 1.00
[INFO] [1751484202.023029025] [waypoints_node]: Path: x: 1.16 y: 1.20
[INFO] [1751484202.023037614] [waypoints_node]: Path: x: 0.80 y: 1.24
[INFO] [1751484202.023044890] [waypoints_node]: Path: x: 0.50 y: 1.24
[INFO] [1751484202.023071901] [waypoints_node]: Path: x: -0.50 y: 1.25
[INFO] [1751484202.023083737] [waypoints_node]: Path: x: -1.28 y: 1.25
[INFO] [1751484202.023109298] [waypoints_node]: Path: x: -1.28 y: 0.76
[INFO] [1751484202.023136160] [waypoints_node]: Path: x: -0.80 y: 0.74
[INFO] [1751484202.023163293] [waypoints_node]: Path: x: -0.46 y: 0.73
```

Figura 1 Consola al ejecutar waypoints\_node

Este nodo leerá el archivo path.yaml (ubicado en trabajo\_pkg/config/) y publicará los waypoints en el tópico /waypoints.

4. **Iniciar la Simulación en CoppeliaSim:** Una vez que todos los nodos estén corriendo, ve a la ventana de CoppeliaSim y haz clic en el botón de "Play" (el triángulo azul) para iniciar la simulación. Los robots deberían comenzar a moverse siguiendo la lógica de relevos.

## DESCRIPCIÓN TÉCNICA

### Duplicar robots en la escena:

Para simular la carrera de relevos con dos robots, se han duplicado los modelos de Turtlebot3 dentro de la escena de CoppeliaSim. Cada instancia del robot en CoppeliaSim se configura para interactuar con ROS2 de forma independiente. Esto se logra asignando un namespace único a cada robot en el archivo relevos\_launch.py. Por ejemplo, un robot operará bajo el namespace /robot1 y el otro bajo /robot2. Esto asegura que cada robot tenga sus propios tópicos de odometría, comandos de velocidad, etc., evitando conflictos y permitiendo un control individualizado.

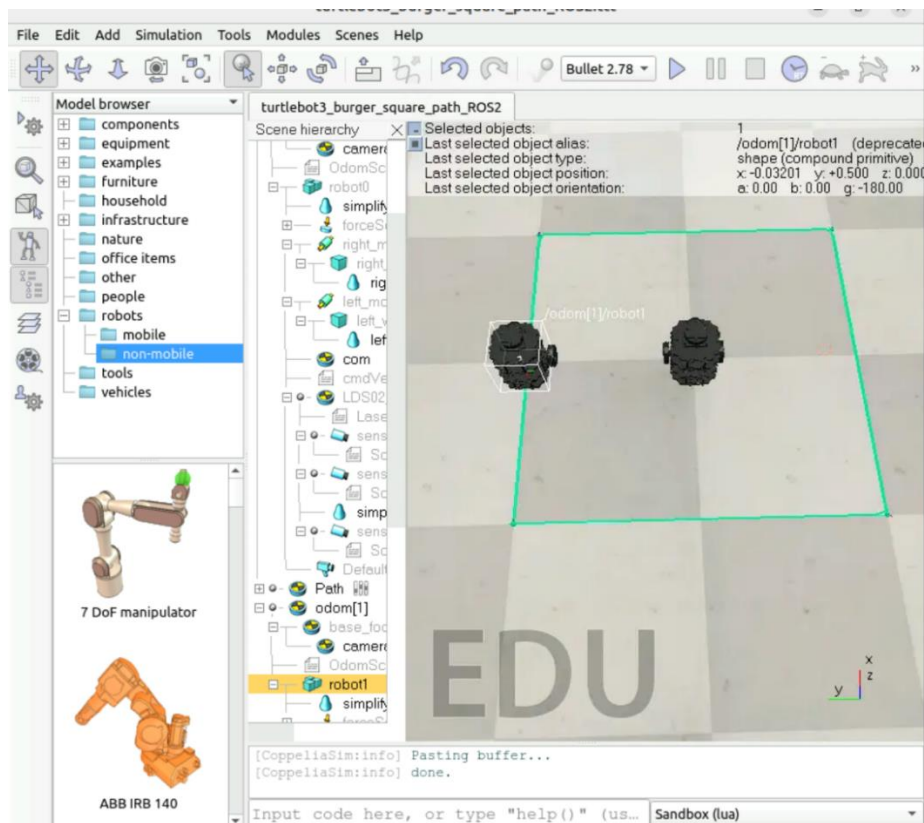
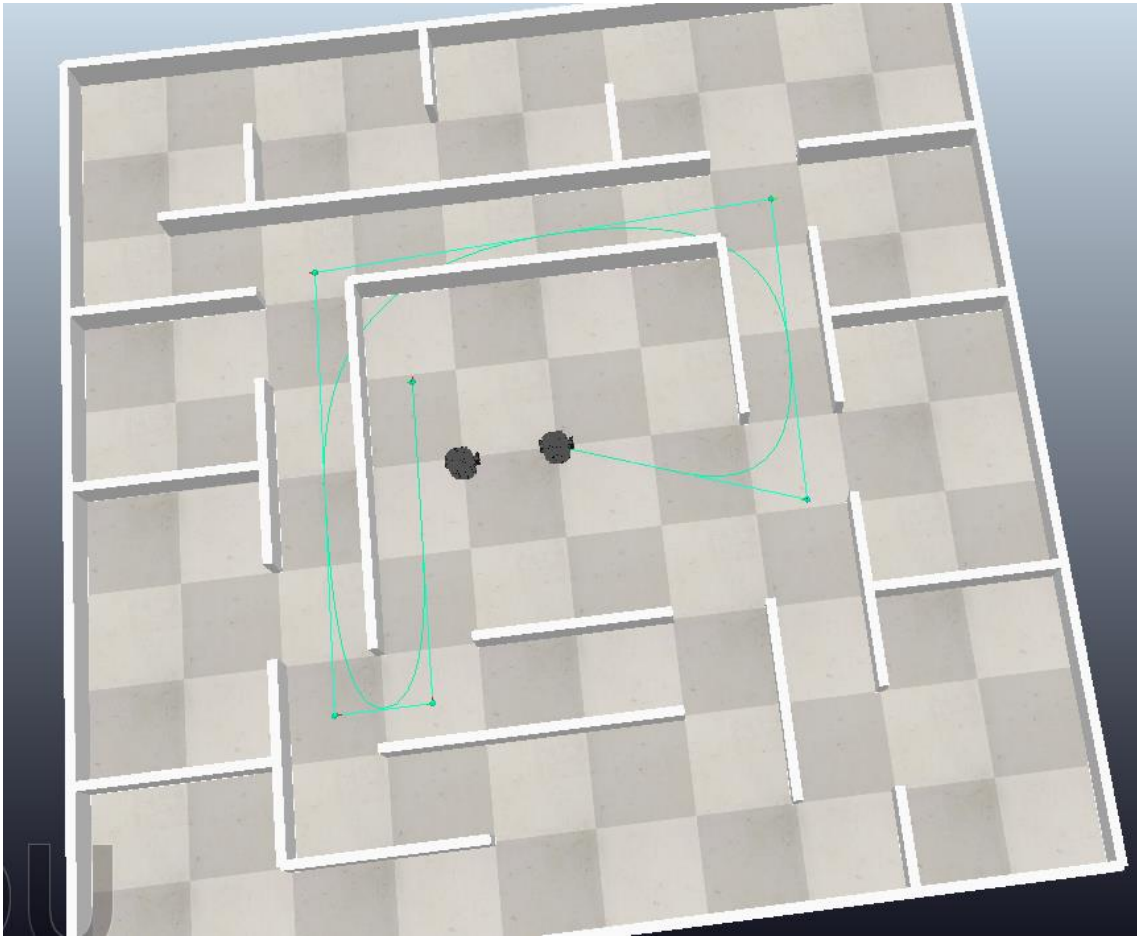


Figura 2 Robots clonados



*Figura 3 Robots en la escena*

### **Remapeo de tópicos:**

El remapeo de tópicos es esencial en un sistema multi-robot para asegurar que cada robot se comunique a través de sus propios canales.

# Para ROBOT1

```
remappings=[ ('odom', '/odom1'),
```

```
# El nodo pure_pursuit_odom de robot1 leerá de /odom1 ('cmd_vel', '/cmd_vel1'),
```

```
# El nodo pure_pursuit_odom de robot1 publicará a /cmd_vel1 ]
```

# Para ROBOT2 remappings=[ ('odom', '/odom2'),

```
# El nodo pure_pursuit_odom de robot2 leerá de /odom2 ('cmd_vel', '/cmd_vel2'),
```

```
# El nodo pure_pursuit_odom de robot2 publicará a /cmd_vel2 ]
```

### **Inspección del sistema:**

ROS2 proporciona varias herramientas de línea de comandos para inspeccionar el estado del sistema, lo cual es invaluable para la depuración y el entendimiento del grafo de computación.

- **ros2 node list:** Muestra una lista de todos los nodos de ROS2 que están actualmente activos en el sistema. Esto te permite verificar que

pure\_pursuit\_robot1, pure\_pursuit\_robot2, relay\_manager, waypoints\_node, y los nodos de CoppeliaSim están corriendo.

- **ros2 topic list:** Enumera todos los tópicos de ROS2 disponibles. Es útil para ver qué información se está compartiendo y bajo qué nombres (ej. /odom1, /cmd\_vel1, /relevo\_active\_robot1, /waypoints).
- **ros2 topic echo /robot1/cmd\_vel:** Muestra el contenido de los mensajes que se publican en un tópico específico en tiempo real. Puedes usarlo para verificar que los comandos de velocidad se están enviando correctamente al robot 1.
- **ros2 topic info /robot1/odom:** Proporciona información detallada sobre un tópico, incluyendo su tipo de mensaje

## VALIDACIÓN Y PRUEBAS

- **rqt\_graph:** para visualizar la red de nodos y tópicos

rqt\_graph

Ejecutar rqt\_graph en una terminal (después de lanzar el relevos\_launch.py y waypoints\_node) te permitirá visualizar el grafo de computación de ROS2. Este grafo muestra los nodos (cuadrados) y los tópicos (círculos) y cómo se conectan entre sí.

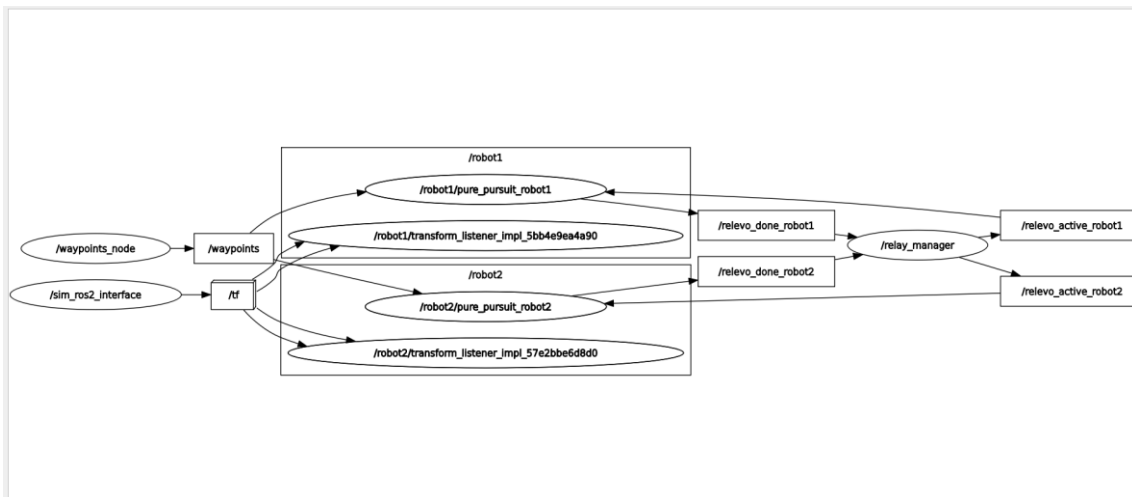
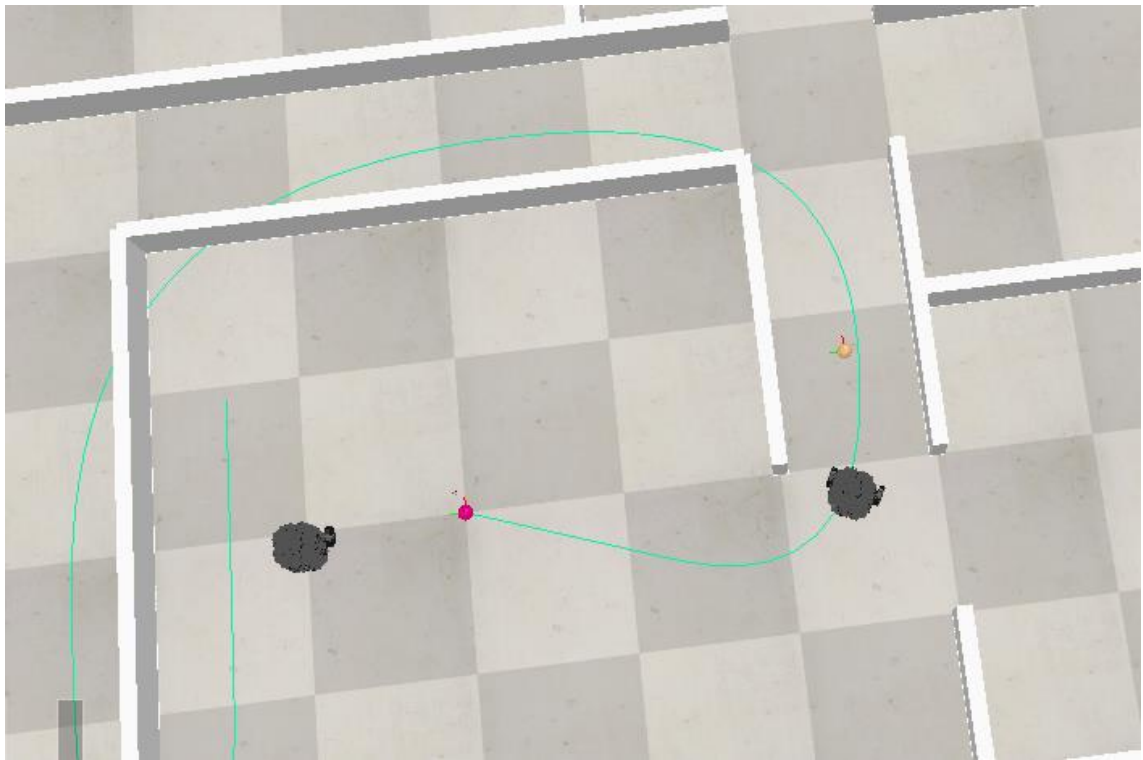


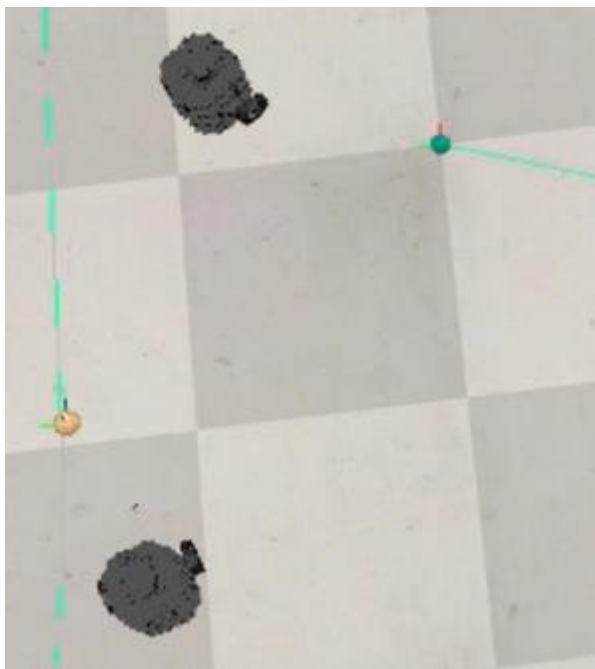
Figura 4 Rqt\_graph con los nodos y topics

Finalmente, una vez se compila todo y se da al botón de play, el robot debería empezar a seguir los waypoints y una vez el robot1 llega al último waypoint el segundo robot comenzará la ruta.





*Figura 5 Robot1 moviéndose por la ruta*



*Figura 6 Robot2 comenzando a moverse tras el relevo*